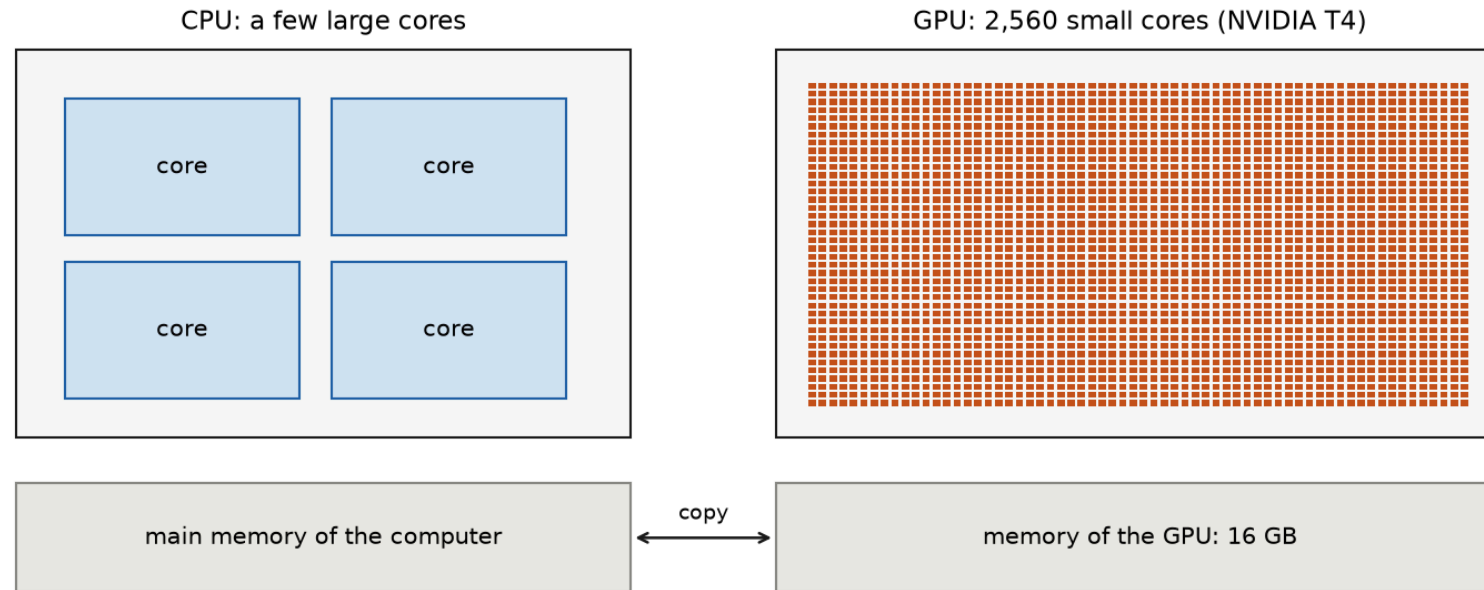


CPU and GPU

The two processors that carry out your products: what is inside each, how one product is divided on each, and why the data is copied.

Two processors

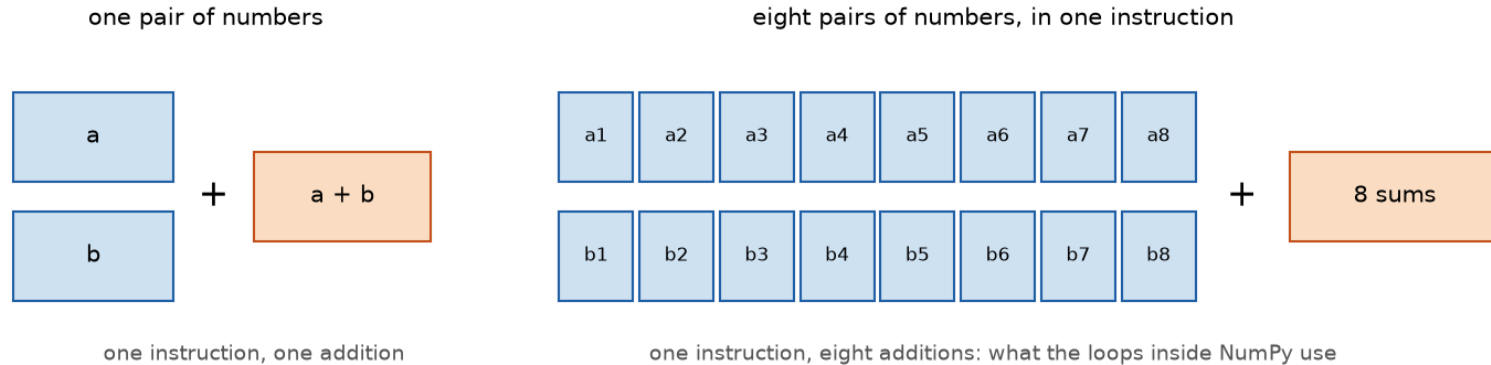


- **CPU:** a few large cores. Each runs any program, one instruction after another, billions a second
- **GPU:** thousands of small cores. The same instruction at the same moment, each core on its own numbers
- Each processor computes with the numbers of its own memory

The two processors, side by side

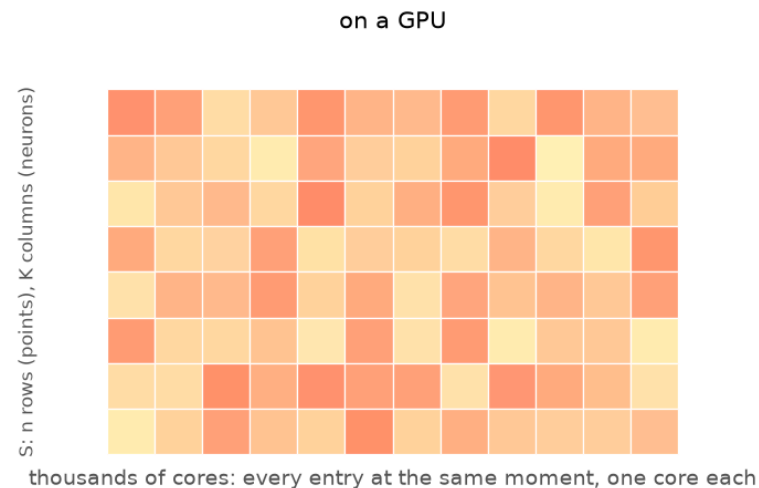
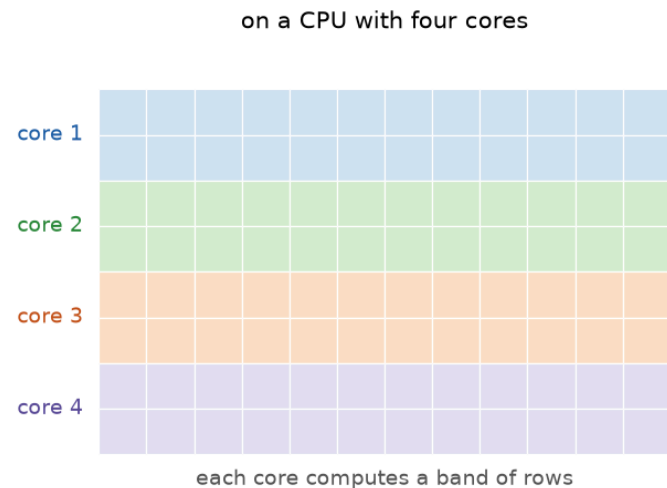
	CPU	GPU
cores	a few: 2 on Colab's free machines, 4 to 16 in a laptop, over 100 in a large server	thousands: 2,560 in the T4
one core	fast; runs any program by itself	simple; works in a group that carries out one instruction at a time
memory	the main memory of the computer: about 12 GB on Colab	its own: 16 GB on the T4
good at	every kind of program, also one whose steps depend on each other	the same operation on very many numbers
name in PyTorch	"cpu"	"cuda"

Inside a CPU core



- One instruction: take it, read its numbers, compute, store the result
- **Several numbers per instruction:** 8 pairs at once, on some processors 16. NumPy's loops use them; Python's interpreter cannot
- **The cache:** a small, fast memory of the core. Numbers that stay in it are read fast
- **Several cores:** NumPy and PyTorch divide a large product among them

One product, divided

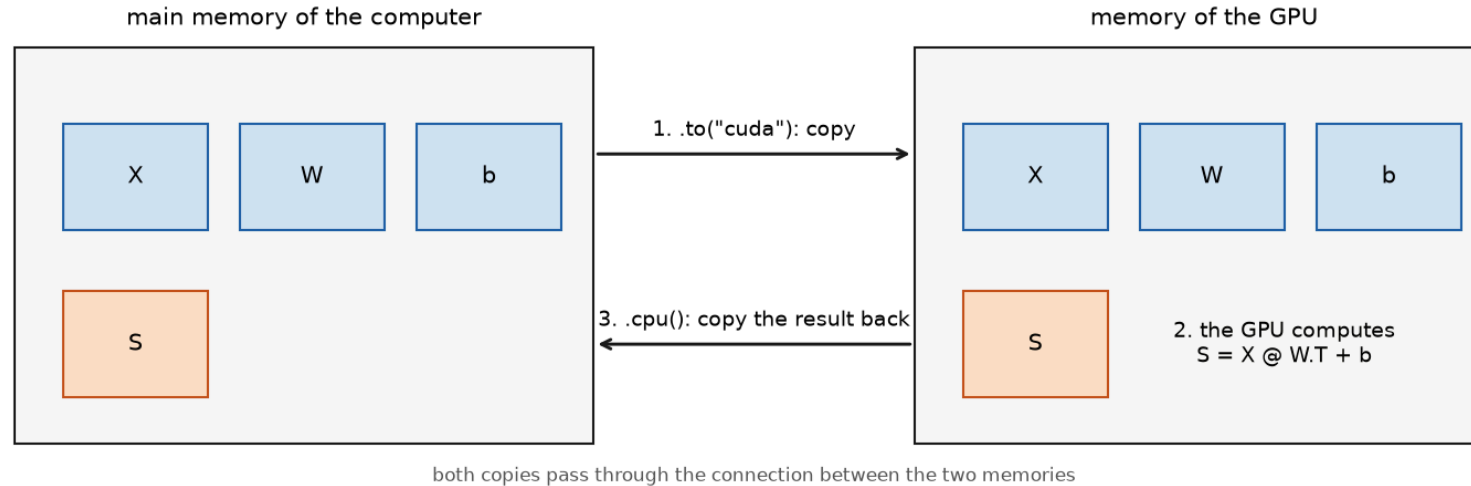


- $S = X @ W.T + b$: $n \times K$ entries, each a sum of m products
- No entry needs another: the work divides in any way among any number of cores
- A CPU: each core takes a band of rows. A GPU: thousands of entries at the same moment

What cannot be divided

- A step that needs the result of the step before it
- Training a network: every step starts from the weights the last step produced
- Inside one step the products divide among the cores; the steps themselves do not

Two memories



- A GPU computes only with numbers in its own memory
- 1. `.to("cuda")` : copy. 2. compute. 3. `.cpu()` : copy the result back
- The copies pass through the connection between the two memories, and take time

When a GPU does not pay

- **Data used once:** copied there and back for one short operation, the copies cost more than the operation
- **Data that does not fit:** the tensors must fit the GPU's memory, 16 GB on the T4
- **Steps that depend on each other:** thousands of cores do not help when the work is one step after another

Where the numbers come from

	NVIDIA T4, from its data sheet	Colab's free machine, October 2026
cores	2,560	2
memory	16 GB, 320 GB/s	about 12 GB
at peak	8.1 trillion operations a second	
power	70 watts	

- Your notebook prints what your own session has
- The page **CPU and GPU** has the same four parts, with the pictures: ikoutis.github.io/cs301-labs/unit-2/pages/cpu-and-gpu.html